

Buna ziua stimati membri ai comisiei, ma numesc... iar in aceasta prezentare voi vorbi despre un protocol de autentificare OAuth folosind Schnorr

Slide 2 - Sumar

În introducere, voi expune tema și scopurile lucrării

Apoi tehnologiile și Arhitectura generală a sistemului

Date despre schema de identificare Schnorr

Fluxurile de înregistrare și autentificare alături de Proprietăți

În final avem rezultatele și concluziile

Slide 3 - Introducere

Tema este data de vulnerabilitățile specifice aplicațiilor web care folosesc un protocol de autentificare OAuth unde se transmite un secret direct fără criptare către serverul de autentificare. Deși protocolul HTTPS reduce riscul interceptării, acesta nu elimină vulnerabilități precum breșele bazelor de date, atacuri de tip replay sau de tip Store Now, Decrypt Later.

Astfel, soluția propusă constă în proiectarea unui protocol de autentificare în care parola nu este transmisă niciodată către server iar prin integrarea schemei de identificare Schnorr am putea folosi zero knowledge proofs pentru a elimina și evita câteva vulnerabilități.

De asemenea s-a dezvoltat și un formular standard de autentificare pe care îl poate integra o aplicație web.

Slide 4 - Tehnologii

Pentru dezvoltarea server-ului s-a ales Python cu Flask (datorită flexibilității și suportului cu o multitudine de biblioteci)

Iar pe partea de client avem JavaScript, HTML și CSS (au fost utilizate pe partea de client, deoarece oferă compatibilitate cu majoritatea framework-urilor folosite în aplicații web.)

Slide 5 - Arhitectură

Arhitectura este pe trei niveluri.

Nivelul de prezentare reprezintă interfața în care userul introduce credențialele și inițiază autentificarea

Nivelul de aplicație conține logica de verificare și autentificare,

Nivelul de date conține bazele de date cu asocierea dintre user, secretul public și resurse

Slide 6 - Algoritmul Schnorr

Schema interactivă Schnorr se bazează pe problema logaritmului discret și este utilizată în domenii precum IoT și blockchain, aceasta este un algoritm de identificare în care un "Prover" își demonstrează identitatea în fața unui "Verifier" printr-un schimb secvențial de mesaje care se termină cu o validare matematică

Slide 7 - Fluxul de înregistrare

Astfel, în implementare, protocolul începe cu înregistrarea unde userul introduce credențialele, aplicația obține parametrii globali de la server-ul de autentificare, derivează x folosind parola, creează cheia publică y și o trimite la server care pe urmă își face verificările propuse și salvează legătura dintre y și utilizator.

Slide 8 - Autentificare (faza commit)

La autentificare, după introducerea credențialelor și schimbul de parametrii, aplicația client generează un nonce aleatoriu, calculează angajamentul cu el și îl transmite împreună cu id-ul clientului.

Serverul validează payload-ul, creează o sesiune temporară din care e creată provocarea și o returnează împreună cu session id.

Slide 9 - Autentificare (faza verify)

Mai departe, în faza de verificare se calculează soluția folosind parametrul derivat, și îl transmite ca răspuns la provocare. Se face egalitatea finală și dacă e corectă, atunci se emite un token

Slide 10 - Consum token

Pe care, aplicația îl poate folosi ca să acceseze resurse protejate de la serverul de resurse.

Slide 11 - Proprietăți de securitate

Putem observa astfel că implementarea previne vulnerabilitățile menționate prin stocarea exclusivă a cheii publice, folosirea valorilor efemere și netransmiterea lui x și r .

Slide 12 - Validare și testare

Validarea acoperă 49 de teste, toate cu rată de succes 100%, unde

5 teste pozitive verifică scenarii de bază/happy paths de autentificare/inregistrare, absența parolei hashuite/criptate în baza de date, funcționarea concurența de utilizatori;

5 teste negative care acoperă autentificarea cu parolă greșită, expirarea sesiunii și diferite cereri duplicate;

31 de corner cases care testează valori în afara spațiului Q^* , valori triviale pentru y și t , tipuri de date neconforme precum float, string sau null, și absența antet;

8 teste de securitate care validează erori off-by-one, izolarea sesiunilor și autentificarea cu cheia publică furată.

Din simulările de atacuri observăm că

Din 10k de cereri nu au fost coliziuni pe c

injectarea de parametri slabi e respinsă prin HTTP 422 (pentru subgrup invalid)

replay cross-sesiune ajunge sa fie respins datorita valorilor efemere

un brute force devine nefezabil dupa o cheie de 64 de biti

Slide 13 - Evaluarea performanței

Pentru implementare s-a folosit Un P de 2048 respectând recomandările NIST

De aceea putem observa că operațiile ce includ exponentiere modulară au un factor de timp mare, operația de validare având cea mai mare latență deoarece are 2 exponențieri

Slide 14 - Comparatie OAuth față de ZKP

Dacă e să comparăm cu un protocol standard OAuth 2.0, vedem cel mai mare minus al protocolului și anume latența

În primul tabel avem măsurători secvențiale pentru diferite implementări, prima ar fi schnorr cu calcule exponențiale având o medie de 64 ms, mai jos avem implementări de oauth2.0 simplu și cu pixy unde avem 2 ms pe autentificare și ultima e o librărie optimizată având 0.8 ms

În testele de debit concurrent realizate cu Locust pe 8 threaduri timp de 60 de secunde, pentru schnorr putem vedea 19 cereri pe sec pentru 10 utilizatori concurenți și 17 cereri pentru 100 utilizatori pe când oauth2.0 are între 440 și 500

Aceste comparații nu sunt perfect echivalente având în vedere calculele pe care le face schnorr și pe care le fac protocoalele oauth

Slide 15 - Comparatie cu protocol SRP

Așa că s-a optat pentru compararea cu alt protocol bazat pe zero knowledge proof cum ar fi SRP, Pentru o comparație fidelă a trebuit executat schnorr de 2 ori deoarece srp are o autentificare mutuală

Știind asta, era de așteptat să vedem timpi foarte mari la calculele exponențiale și am trecut pe curbe eliptice ajungând la 38 ms

În final s-a adăugat și o variantă de schnorr pe curbe eliptice dintr-o librărie ce folosește cod C și s-a ajuns la un total de 9.5 ms

Slide 16 - Concluzii

În concluzie schema Schnorr poate fi integrată cu succes într-un protocol de tip Open Authorization, din urma căruia un utilizator se poate autentifica și obține un token spre utilizare ulterioară anulând efectele unei breșe a bazei de date sau ale unei exfiltrări a datelor prin folosirea valorilor efemere și prin nepartajarea parolei către serverul de autentificare.

Ca limitări se remarcă latența crescută care ar putea fi redusă prin adăugarea curbelor eliptice și prin folosirea unor limbaje care permit un grad sporit de optimizare iar verificarea la browser poate fi indeplinită printr-o combinație a protocoalelor OAuth2.0 cu schnorr sau un schnorr mutual

Slide 17 - Bibliografie selectivă

Acestea sunt câteva referințe din bibliografia utilizată.

Slide 18 - Mulțumesc

Vă mulțumesc pentru atenție!